1. Question

Write a Java program based on the given UML class diagram to implement an Order Management System.

The system should include:

Customer class with name, location, sendOrder() and receiveOrder().

Order class with date, number, confirm() and close().

SpecialOrder and NormalOrder classes inheriting from Order.

Implement the respective dispatch() and receive() methods.

2. Steps for Execution

1.Create Main.java.

2.Write the given Java program.

3.Compile:

javac Main.java

4.Run:

- java Main
- The program creates Customer, SpecialOrder, and NormalOrder objects.

5.It displays customer and order details and performs order operations.

3. Algorithm

1.Start.

2.Create Customer class with name, location, and order methods.

3.Create Order class with date, number, confirm and close methods.

4.Create SpecialOrder and NormalOrder by extending Order.

5.Create objects for customer and both orders.

6.Display customer details.

7.Confirm, close, and dispatch orders.

8.Receive the normal order.

9.Display the results.

10.Stop.

UML CLASS DIAGRAM:

```
┌───────────────────────┐
|     Customer      |
├───────────────────────┤
| name : String      |
| location : String   |
├───────────────────────┤
| sendOrder()        |
| receiveOrder()      |
└───────────┬───────────┘
        |
        ▼
┌───────────────────────┐
|      Order       |
├───────────────────────┤
| date : String      |
| number : String     |
├───────────────────────┤
| confirm()         |
| close()          |
└───────────┬───────────┘
        |
    ┌─────┴─────┐
    ▼      ▼
┌─────────────┐ ┌─────────────┐
```

```
|SpecialOrder| |NormalOrder |
├────────────┤ ├────────────┤
| dispatch() | | dispatch() |
|            | | receive()  |
└────────────┘ └────────────┘
```

CODE:

```
package javacore;
class Customer {
   String name;
   String location;
   void sendOrder() {
      System.out.println(name + " sent the order.");
   }
   void receiveOrder() {
      System.out.println(name + " received the order.");
   }
}
class Order {
   String date;
   String number;
   void confirm() {
      System.out.println("Order confirmed.");
   }
   void close() {
      System.out.println("Order closed.");
   }
}
```

```
class SpecialOrder extends Order {
    void dispatch() {
        System.out.println("Special order dispatched.");
    }
}
class NormalOrder extends Order {
    void dispatch() {
        System.out.println("Normal order dispatched.");
    }
    void receive() {
        System.out.println("Normal order received.");
    }
}
public class Main {
    public static void main(String[] args) {
        Customer c = new Customer();
        c.name = "Sameera";
        c.location = "Vizianagaram";
        SpecialOrder so = new SpecialOrder();
        so.date = "22-09-2026";
        so.number = "S202";
        NormalOrder no = new NormalOrder();
        no.date = "22-09-2026";
        no.number = "N102";
        System.out.println("Customer: " + c.name);
        System.out.println("Location: " + c.location);
        c.sendOrder();
        System.out.println("\nSpecial Order");
```

```java
        System.out.println("Date: " + so.date);
        System.out.println("Number: " + so.number);
        so.confirm();
        so.close();
        so.dispatch();
        System.out.println("\nNormal Order");
        System.out.println("Date: " + no.date);
        System.out.println("Number: " + no.number);
        no.confirm();
        no.close();
        no.dispatch();
        no.receive();
        c.receiveOrder();
    }
}
```

OUTPUT SCREENSHOT (SAMPLE RUN):

Customer: Sameera

Location: Vizianagaram

Reshma sent the order.

Special Order

Date: 22-09-2026.

Number: S202.

Order confirmed.

Order closed.

Special order dispatched.

Normal Order

Date: 22-09-2026.

Number: N102.

Order confirmed.

Order closed.

Normal order dispatched.

Normal order received.

Reshma received the order.